



Porting ZEPHYR RTOS to SHAKTI

SHAKTI Group | CSE Dept | PS-CDISHA - RISE LAB

IIT Madras

OS vs RTOS

What is an OS? “*An OS makes your system smart*”

- ❖ Manages hardware and software resources of a computer or embedded system
- ❖ Ensures multitasking, memory management and process scheduling
example: Linux and Windows

What is an RTOS? “*An RTOS makes your system punctual*”.

- ❖ An OS designed for real-time and deterministic response
- ❖ Guarantees that tasks meet strict timing deadlines. Ideal for time-critical embedded applications
- ❖ Example: Zephyr, FreeRTOS, VxWorks, etc

OS focuses on efficiency — RTOS focuses on timing

What is ZEPHYR?

- ❖ **Best of breed RTOS-The power of Linux — scaled down for microcontrollers and IOT devices.**
- ❖ **A proven RTOS ecosystem — built by developers, for developers — supporting multiple architectures (ARM, RISC-V, x86, and more).**
- ❖ **An open-source RTOS built to run efficiently on MCUs for wearables and IoT — where silicon cost is minimal and performance matters most.**

Why Choose Zephyr?

ZEPHYR

Tiny. Fast , where open source meets real-time precision.

- ❖ **Small Footprint** – Runs on MCUs with as little as 8 kB RAM
- ❖ **Highly Modular** – Enable only required subsystems
- ❖ **Scalable** – From simple SoCs to advanced IoT devices
- ❖ **Rich Ecosystem** – Networking, security, and device drivers
- ❖ **Cross-Platform** – Uniform APIs for different architectures
- ❖ **Developer-Friendly** – west, CMake, and Device tree support
- ❖ **Strong Community** – Maintained by global contributors
- ❖ **Cross-architecture**- support for diverse MCUs



ZEPHYR FOLDER STRUCTURE

layer	Folder / Path	Description	Key Components / Purpose
Core Kernel	kernel/	Core RTOS functionality	Scheduler, threads, semaphores, timers; handles task management and context switching
Hardware Abstraction	arch/	Architecture-specific support	Startup assembly, context switch, ISR stubs for RISC-V, ARM, x86, etc.
	soc/	SoC-specific support	Contains subfolders for each SoC; defines memory map, peripherals, and startup configuration
	boards/	Board-specific configuration	DTS files, Kconfig, and board.cmake; defines board-level hardware and build info
	dts/	Device Tree Sources and bindings	.dts, .dtsi, .yaml files describe hardware and generate device tree headers at build time
Drivers	include/	Public headers and APIs	zephyr.h, kernel.h, and driver headers under include/drivers/; used by apps/subsystems
	drivers/	Hardware drivers	UART, SPI, I2C, GPIO, pinctrl; provides Zephyr device driver API implementations
Application	samples/, tests/	Applications and tests	Example apps (hello_world, blinky) and system/unit test cases
Build & Tools	cmake/	Build system support	CMake helper modules and configuration logic

Zephyr Samples Overview

- ❖ Zephyr comes with ready-to-run sample applications to verify board bring-up, driver functionality, and OS services. Samples are located in `zephyr/samples`.
- ❖ Each sample includes ready-to-build source code, CMake configuration, and a README describing usage and expected output.

Sample Name	Path	Description
Hello World	<code>samples/hello_world</code>	Prints “Hello World! <board>” via UART – basic sanity check.
Blinky	<code>samples/basic/blinky</code>	Toggles an LED using GPIO – tests GPIO driver
Synchronization	<code>samples/kernel/synchronization</code>	Demonstrates threads and semaphores in Zephyr kernel.

Steps for Porting Zephyr on Shakti SoC

- 1. Create Custom Board Directory**
- 2. Define SoC Support**
- 3. Write Device Tree (DTS) Files**
- 4. Implement Drivers**

Custom Board Support – Shakti (Zephyr)

Create Custom Board Directory

Copy a reference board (e.g., boards/riscv/sifive_e)

Paste and rename to boards/riscv/shakti/

Add Required Board Files

board_defconfig – default configuration

board.yaml – board metadata and properties

Kconfig.board – board-specific configuration options

CMakeLists.txt / board.cmake – build system integration

Verify Board Registration - Ensure Zephyr recognizes the new board

Check detection using: **west boards | grep shakti**

Tip: Use an existing RISC-V board (like sifive_e or qemu_riscv32) as a template for faster integration.

Defining SoC Support – Shakti (Zephyr)

Create SoC Directory

Path: soc/riscv/shakti/ Base it on an existing RISC-V SoC (e.g., sifive/)

Add Required SoC Files

soc.h – SoC-specific definitions (registers, addresses)

Kconfig.soc – configuration options

Kconfig.defconfig – default SoC settings

soc.yml – metadata and description

CMakeLists.txt – build integration

Integrate & Verify

Ensure correct linkage between board and SoC layers

Tip: Use an existing RISC-V SoC (like sifive) as a reference for directory structure and configuration style.

Writing Device Tree (DTS) for Shakti (Zephyr)

Add Device Tree Files

Create shakti.dts (board-level)

Include SoC-level file: `#include "shakti.dtsi"`

Define Hardware Components

CPU: Type, ISA, clock frequency

Memory: Base address, size

Peripherals: UART, SPI, I2C, GPIO, timers

Validate DTS Integration

Ensure correct syntax and labels (`&uart0`, `&gpio0`, etc.)

Tip:

Reuse DTS structure from an existing RISC-V board and modify nodes for Shakti peripherals.

Start only with the UART peripheral during initial porting — this avoids build and compile errors and simplifies debugging and output verification.

Later, add other peripherals (SPI, I2C, GPIO, etc.) as required.

Implementing Driver for Shakti UART

Create Driver Directory

Path: drivers/serial/uart_shakti.c

Base it on an existing UART driver (e.g., uart_sifive.c)

Implement or modify Basic UART Functions

Initialize UART hardware

Configure baud rate and data format

Implement transmit and receive routines

Register with Zephyr

Add driver entry in CMakeLists.txt under drivers/serial/

Update Kconfig to enable CONFIG_UART_SHATKI option

Enable in Board Configuration- In board_defconfig(shakti_defconfig)

CONFIG_SERIAL=y

CONFIG_UART_CONSOLE=y

CONFIG_UART_SHATKI=y

Tip: Start driver support with UART only to ensure basic serial output and debugging functionality. Add other drivers (SPI, I2C, GPIO) after successful UART testing.

Run, Test & Debug on Shakti

Configure & Build

west build -b shakti samples/hello_world

Successful build generates an ELF binary (zephyr.elf) that includes: Zephyr kernel, Board-specific drivers (UART, GPIO, etc.), Application (Hello World)

6. Test on QEMU & Hardware (FPGA/ASIC):

QEMU - Used for early testing without hardware.

FPGA Hardware Test - Flash the same ELF to the Shakti FPGA board.

7. Debug & Validate

GDB for step-by-step code tracing

UART logs for runtime analysis

Goal: Validate OS boot, driver initialization, and “Hello World” output

Outcome: Successful execution on both QEMU and physical Shakti confirms complete Zephyr integration and porting.

output

QEMU OUTPUT

```
(.venv) suganthi@sjd:~/zephyrproject/zephyr/samples/hello_world$ west build -t run -d build_qemu
-- west build: running target run
[0/1] To exit from QEMU enter: 'CTRL+a, x'[QEMU] CPU: riscv32
*** Booting Zephyr OS build v4.1.0-6396-g92d4669752b6 ***
Hello World! qemu_riscv32/qemu_virt_riscv32
█
```

FPGA OUTPUT

```
suganthi@sjd:/scratch2/backup_24_01/Videos$ sudo miniterm.py /dev/ttyUSB4 115200
--- Miniterm on /dev/ttyUSB4: 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
*** Booting Zephyr OS build v4.1.0-6396-g92d4669752b6 ***
Hello World! shakti/shakti
```

Zephyr RTOS – Official Resources and Guides

- ❖ For complete environment setup and build instructions,
- ❖ please refer to the “Zephyr Environment Setup & Build Guide” document.
- ❖ Documentation: <https://docs.zephyrproject.org/latest/>
- ❖ Website <http://www.zephyrproject.org/>
- ❖ Zephyr source (GitHub): <https://github.com/zephyrproject-rtos/zephyr>
- ❖ Supported Boards: <https://docs.zephyrproject.org/latest/boards/>
- ❖ Samples and Demos: <https://docs.zephyrproject.org/latest/samples>
- ❖ Getting Started Guide:
https://docs.zephyrproject.org/latest/develop/getting_started/

Queries ?



Thank you!

Website: shakti.org.in

